

计算机网络实验报告

课程名称计算机网络成绩评定

实验项目名称Socket 通信综合实验指导教师张伟

实验项目编号实验 7实验项目类型实验地点滢昌苑 D 座 111

学生姓名李文峰学号202300460158

学院网络空间安全专业网络空间安全

实验时间2025 年 5 月 7 日

一、实验目的

1. 理解基于 TCP 协议的 Socket 通信机制；
2. 掌握 Socket 编程模型及其在客户端和服务端的基本应用；
3. 掌握多线程技术在 Socket 服务器中处理并发客户端连接的实现方式；
4. 掌握进程间通信中共享资源的同步机制。

二、实验步骤与结果

任务一：基础 Socket 通信实验

代码测试结果如下图所示：

CLIENT.py

```
1 import socket
2
3 client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
4
5 server_address = ("localhost", 8888)
6 client_socket.connect(server_address)
7
8 try:
9     message = "Hello, server! I am client. Form 202300460158"
10    client_socket.sendall(message.encode())
11
12    data = client_socket.recv(1024)
13    print(f"收到来自服务器的数据: {data.decode()}")
14
15 finally:
16    client_socket.close()
```

运行 CLIENT

C:\Users\30545\PycharmProjects\TCP_CLIENT\.venv\Scripts\python.exe C:\Users\30545\PycharmProjects\TCP_CLIENT\CLIENT.py

收到来自服务器的数据: Hello, client! I am server. From 202300460158

进程已结束，退出代码为 0

server.py

```
8 server_socket.listen(1)
9 print("等待客户端连接")
10 connection, client_address = server_socket.accept()
11
12 try:
13     print(f"连接来自于 {client_address}")
14
15     data = connection.recv(1024)
16     print(f"接收到的客户端数据为: {data.decode()}")
17
18     message = "Hello, client! I am server. From 202300460158"
19     connection.sendall(message.encode())
20
21 finally:
22     connection.close()
23     server_socket.close()
```

运行 server

C:\Users\30545\PycharmProjects\TCP\.venv\Scripts\python.exe C:\Users\30545\PycharmProjects\TCP\server.py

等待客户端连接

连接来自于 ('127.0.0.1', 51964)

接收到的客户端数据为: Hello, server! I am client. Form 202300460158

进程已结束，退出代码为 0

1. 如果服务器未启动时客户端尝试连接会发生什么，以及为什么会这样？

当服务器未启动时客户端尝试连接，会导致客户端连接失败出现报错，如下图所示。这是因为 TCP 是一种面向连接的协议，客户端通过 `connect()` 发起连接请求，尝试与服务器建立三次握手，服务器必须在指定的地址和端口上绑定并监听才能接受连接。如果服务器未启动，没有程序监听 8888 端口，客户端的连接请求则无法被响应。

```
C:\Users\30545\PycharmProjects\TCP_CLIENT\.venv\Scripts\python.exe
C:\Users\30545\PycharmProjects\TCP_CLIENT\CLIENT.py
Traceback (most recent call last): 使用 AI 解释
  File "C:\Users\30545\PycharmProjects\TCP_CLIENT\CLIENT.py", line 6, in <module>
    client_socket.connect(server_address)
ConnectionRefusedError: [WinError 10061] 由于目标计算机积极拒绝，无法连接。

进程已结束，退出代码为 1
```

2. `recv(1024)` 中的 1024 表示什么含义？是否保证一次读取 1024 字节？

`Recv (1024)` 中的 1024 表示客户端或服务器在调用 `recv()` 时，分配的最大缓冲区大小为 1024 字节，即一次调用时能够接收的最大数据量。

但 `recv(1024)` 并不保证一次读取正好 1024 字节，`recv(1024)` 会返回实际接收到的数据，若对方发送的数据小于 1024 字节则只返回实际的数据，若大于 1024 字节则只接收 1024 字节。

3. 为什么服务器代码中需要先调用 `accept()` ？

`accept()` 的作用是从服务器的监听队列中取出客户端的连接请求，建立与客户端的通信通道。在三次握手时，客户端通过 `connect()` 发起连接请求、服务器的 `listen()` 使套接字进入监听状态，接受客户端的连接请求并将其放入队列、最后 `accept()` 从队列中取出连接请求完成三次握手。所以，先调用 `accept()` 是为了完成第三次握手，以实现后续的信息传输。

任务二：多线程 Socket 通信实验

下图为服务器与五个客户端建立连接并发送消息的过程

```
C:\Users\30545\PycharmProjects\TCP\.venv\Script
服务器监听端口60000中...
```

```
[连接] 与客户端('127.0.0.1', 54834)建立连接
[连接] 与客户端('127.0.0.1', 54837)建立连接
[连接] 与客户端('127.0.0.1', 54839)建立连接
[连接] 与客户端('127.0.0.1', 54844)建立连接
[连接] 与客户端('127.0.0.1', 54845)建立连接
```

建立连接

接着客户端 1、2、3 分别发送消息，客户端和服务端上的显示如下所示

```
C:\Users\30545\PycharmProjects\TCP\.venv\Scripts\p C:\Users\30545\PycharmProjects\TCP\.venv\Script:
[连接] 客户端1 已连接至 ('127.0.0.1', 60000) [连接] 客户端2 已连接至 ('127.0.0.1', 60000)
请输入要发送的消息，或输入quit断开连接 请输入要发送的消息，或输入quit断开连接
[发送] 客户端1 发送消息: 1111 [发送] 客户端2 发送消息: 9999
[接收] 来自服务器的回复: 服务器已收到, counter值为 1 [接收] 来自服务器的回复: 服务器已收到, counter值为 4
[发送] 客户端1 发送消息: 5555 [发送] 客户端2 发送消息:
[接收] 来自服务器的回复: 服务器已收到, counter值为 2
[发送] 客户端1 发送消息: 3333
[接收] 来自服务器的回复: 服务器已收到, counter值为 3
[发送] 客户端1 发送消息: |
```

```
C:\Users\30545\PycharmProjects\TCP\.venv\Scripts
服务器监听端口60000中...
[连接] 与客户端('127.0.0.1', 54834)建立连接
[连接] 与客户端('127.0.0.1', 54837)建立连接
[连接] 与客户端('127.0.0.1', 54839)建立连接
[连接] 与客户端('127.0.0.1', 54844)建立连接
[连接] 与客户端('127.0.0.1', 54845)建立连接
[接收] 接收到来自('127.0.0.1', 54834)的消息
[接收] 接收到来自('127.0.0.1', 54834)的消息
[接收] 接收到来自('127.0.0.1', 54834)的消息
[接收] 接收到来自('127.0.0.1', 54837)的消息
[接收] 接收到来自('127.0.0.1', 54839)的消息
```

```
C:\Users\30545\PycharmProjects\TCP\.venv\Script:
[连接] 客户端3 已连接至 ('127.0.0.1', 60000)
请输入要发送的消息，或输入quit断开连接
[发送] 客户端3 发送消息: 222
[接收] 来自服务器的回复: 服务器已收到, counter值为 5
[发送] 客户端3 发送消息: |
```

最后客户端 1、2、3 主动断开连接，服务器显示如下图所示

```
[接收] 接收到来自('127.0.0.1', 54839)的消息
[接收] 接收到来自('127.0.0.1', 54834)的消息
[断开] 客户端('127.0.0.1', 54834)断开连接
[接收] 接收到来自('127.0.0.1', 54837)的消息
[断开] 客户端('127.0.0.1', 54837)断开连接
[接收] 接收到来自('127.0.0.1', 54839)的消息
[断开] 客户端('127.0.0.1', 54839)断开连接
```

实验步骤：

1.编写 Server，代码如下：

在服务器代码中，其监听 8000 端口，当有客户端与其进行连接时，创建一个专属于该客户端的线程以进行通信，当接收到客户端消息时，服务器将计数器 counter 上锁并将其加 1，然后将计数器的值返回给客户端，直到客户端主动断开连接。

```
1. import socket
2. import threading
3.
4. # Global counter
5. counter = 0
6. lock = threading.Lock()
7.
8. def handle_client(client_socket, address):
9.     global counter
10.    print(f"[连接] 与客户端{address}建立连接")
11.    try:
12.        while True:
13.            data = client_socket.recv(1024).decode()
14.            print(f"[接收] 接收到来自{address}的消息")
15.            if not data:
16.                break
17.            with lock:
18.                counter += 1
```

```

19.         response = f"服务器已收到, counter 值为 {counter}"
20.         client_socket.send(response.encode())
21.     except Exception as e:
22.         print(f"Error handling client {address}: {e}")
23.     finally:
24.         print(f"[断开] 客户端{address}断开连接")
25.         client_socket.close()
26.
27. def start_server():
28.     server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
29.     server.bind(('localhost', 60000))
30.     server.listen(5)
31.     print("服务器监听端口 60000 中...")
32.
33.     while True:
34.         client_socket, address = server.accept()
35.         client_thread = threading.Thread(target=handle_client, args=(client_socket, address))
36.         client_thread.start()
37.
38. if __name__ == "__main__":
39.     start_server()

```

编写 Client 程序，代码如下：

客户端与选定的服务器进行连接，然后进行消息的发送，每次发送消息后都会收到服务器的回复，最后可以通过输入 exit 主动断开连接。

```

1. import socket
2.
3. def start_client(client_id):
4.     client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
5.     address= ('127.0.0.1', 60000)
6.     client.connect(address)
7.     print(f"[连接] 客户端{client_id} 已连接至 {address}")
8.     print("请输入要发送的消息，或输入 quit 断开连接")
9.
10.    try:
11.        while True:
12.            message = input(f"[发送] 客户端{client_id} 发送消息: ")
13.            if message.lower() == 'quit':

```



```

14.             break
15.             client.send(message.encode())
16.             response = client.recv(1024).decode()
17.             print(f"[接收] 来自服务器的回复: {response}")
18.
19.
20.     except Exception as e:
21.         print(f"Client {client_id} error: {e}")
22.     finally:
23.         client.close()
24.         print(f"[断开] 客户端{client_id} 与 {address} 断开连接")
25.
26. if __name__ == "__main__":
27.     start_client(1)

```

2.多客户端并发测试

同时打开多个客户端程序（比如 3-5 个），分别运行，结果可以看上图。服务器能够稳定和客户端进行通信，能并发处理多个客户端的通信请求，在每次通信中计数器都加 1，通过计数器的值可以观察到通信的正确性，线程已被成功创建。

3.异常处理测试

当客户端主动输入“quit”进行连接断开时，结果如下所示，服务器能正确关闭相关线程

<pre> C:\Users\30545\PycharmProjects\TCP\.venv\Scripts\ [连接] 客户端1 已连接至 ('127.0.0.1', 60000) 请输入要发送的消息，或输入quit断开连接 [发送] 客户端1 发送消息: quit [断开] 客户端1 与 ('127.0.0.1', 60000) 断开连接 进程已结束，退出代码为 0 </pre>	<pre> C:\Users\30545\PycharmProjects\TCP\.venv\Scripts\ 服务器监听端口60000中... [连接] 与客户端('127.0.0.1', 60543)建立连接 [接收] 接收到来自('127.0.0.1', 60543)的消息 [断开] 客户端('127.0.0.1', 60543)断开连接 </pre>
--	--

当客户端意外断开连接时（即直接关闭程序运行），可以看出服务器仍能够捕获异常并正确关闭相关线程，结果如下所示，第一张图为客户端异常断开连接时的显示，第二张图为服务器在客户端断开后的显示。

```

C:\Users\30545\PycharmProjects\TCP\.venv\Scripts\python.exe C:\Users\30
[连接] 客户端1 已连接至 ('127.0.0.1', 60000)
请输入要发送的消息, 或输入quit断开连接
[发送] 客户端1 发送消息: 【断开】 客户端1 与 ('127.0.0.1', 60000) 断开连接
Traceback (most recent call last): 使用 AI 解释
  File "C:\Users\30545\PycharmProjects\TCP\client.py", line 27, in <mod
    start_client(1)
  File "C:\Users\30545\PycharmProjects\TCP\client.py", line 12, in star
    message = input(f"【发送】 客户端{client_id} 发送消息: ")
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "<frozen codecs>", line 319, in decode
KeyboardInterrupt

进程已结束, 退出代码为 -1073741510 (0xC000013A: interrupted by Ctrl+C)

```

```

C:\Users\30545\PycharmProjects\TCP\.venv\
服务器监听端口60000中...
【连接】 与客户端('127.0.0.1', 60694)建立连接
【接收】 接收到来自('127.0.0.1', 60694)的消息
【断开】 客户端('127.0.0.1', 60694)断开连接
I

```

4.性能测试

在不使用线程锁的情况下, counter 的值出现明显的重复、缺失或错乱的现象

逐步增加客户端数量, 服务器会出现线程数耗尽、阻塞等问题, 如图所示。

```

服务器已收到, counter值为 2632, re
服务器已收到, counter值为 2632, re

```

```

服务器已收到, counter值为 2633, re
服务器已收到, counter值为 2633, re

```

```

服务器已收到, counter值为 2634, re
服务器已收到, counter值为 2634, re

```

问题思考

1.如果多线程服务器在处理 1000 个客户端时可能会有哪些性能瓶颈?

为每个客户端创建一个线程(共 1000 个线程)会导致线程创建和销毁的开销较大。线程切换(上下文切换)也会消耗 CPU 资源, 主要瓶颈包括线程创建/切换开销、系统资源限制、

内存使用、数据竞争、I/O 阻塞等。

2.在多线程环境中多个线程同时访问共享资源可能导致什么问题？该如何解决？

多线程环境中共享资源可能导致数据竞争、数据不一致、死锁和性能瓶颈。解决方法包括使用锁、原子操作、无锁数据结构、避免共享资源，以及切换到异步或多进程模型。

任务三：TCP 协议分析与 Wireshark 抓包

抓包截图如图所示：

tcp.port == 8888						
No.	Time	Source	Destination	Protocol	Length	Info
739	15.353201	127.0.0.1	127.0.0.1	TCP	56	58995 → 8888 [SYN] Seq=2730905436 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
740	15.353275	127.0.0.1	127.0.0.1	TCP	56	8888 → 58995 [SYN, ACK] Seq=252012161 Ack=2730905437 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
741	15.353332	127.0.0.1	127.0.0.1	TCP	44	58995 → 8888 [ACK] Seq=2730905437 Ack=252012162 Win=65280 Len=0
742	15.353418	127.0.0.1	127.0.0.1	TCP	90	58995 → 8888 [PSH, ACK] Seq=2730905437 Ack=252012162 Win=65280 Len=46
743	15.353438	127.0.0.1	127.0.0.1	TCP	44	8888 → 58995 [ACK] Seq=252012162 Ack=2730905483 Win=65280 Len=0
744	15.353609	127.0.0.1	127.0.0.1	TCP	90	8888 → 58995 [PSH, ACK] Seq=252012162 Ack=2730905483 Win=65280 Len=46
745	15.353645	127.0.0.1	127.0.0.1	TCP	44	58995 → 8888 [ACK] Seq=2730905483 Ack=252012208 Win=65280 Len=0
746	15.353681	127.0.0.1	127.0.0.1	TCP	44	8888 → 58995 [FIN, ACK] Seq=252012208 Ack=2730905483 Win=65280 Len=0
747	15.353700	127.0.0.1	127.0.0.1	TCP	44	58995 → 8888 [FIN, ACK] Seq=2730905483 Ack=252012208 Win=65280 Len=0
748	15.353719	127.0.0.1	127.0.0.1	TCP	44	8888 → 58995 [ACK] Seq=252012209 Ack=2730905484 Win=65280 Len=0
753	15.663918	127.0.0.1	127.0.0.1	TCP	44	[TCP Retransmission] 8888 → 58995 [FIN, ACK] Seq=252012208 Ack=2730905484 Win=65280 Len=0
754	15.663953	127.0.0.1	127.0.0.1	TCP	44	[TCP ZeroWindow] 58995 → 8888 [ACK] Seq=2730905484 Ack=252012209 Win=0 Len=0

三次握手的三个包为序号 739、740、741

tcp.port == 8888						
No.	Time	Source	Destination	Protocol	Length	Info
739	15.353201	127.0.0.1	127.0.0.1	TCP	56	58995 → 8888 [SYN] Seq=2730905436 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
740	15.353275	127.0.0.1	127.0.0.1	TCP	56	8888 → 58995 [SYN, ACK] Seq=252012161 Ack=2730905437 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
741	15.353332	127.0.0.1	127.0.0.1	TCP	44	58995 → 8888 [ACK] Seq=2730905437 Ack=252012162 Win=65280 Len=0
742	15.353418	127.0.0.1	127.0.0.1	TCP	90	58995 → 8888 [PSH, ACK] Seq=2730905437 Ack=252012162 Win=65280 Len=46
743	15.353438	127.0.0.1	127.0.0.1	TCP	44	8888 → 58995 [ACK] Seq=252012162 Ack=2730905483 Win=65280 Len=0
744	15.353609	127.0.0.1	127.0.0.1	TCP	90	8888 → 58995 [PSH, ACK] Seq=252012162 Ack=2730905483 Win=65280 Len=46
745	15.353645	127.0.0.1	127.0.0.1	TCP	44	58995 → 8888 [ACK] Seq=2730905483 Ack=252012208 Win=65280 Len=0
746	15.353681	127.0.0.1	127.0.0.1	TCP	44	8888 → 58995 [FIN, ACK] Seq=252012208 Ack=2730905483 Win=65280 Len=0
747	15.353700	127.0.0.1	127.0.0.1	TCP	44	58995 → 8888 [FIN, ACK] Seq=2730905483 Ack=252012208 Win=65280 Len=0
748	15.353719	127.0.0.1	127.0.0.1	TCP	44	8888 → 58995 [ACK] Seq=252012209 Ack=2730905484 Win=65280 Len=0
753	15.663918	127.0.0.1	127.0.0.1	TCP	44	[TCP Retransmission] 8888 → 58995 [FIN, ACK] Seq=252012208 Ack=2730905484 Win=65280 Len=0
754	15.663953	127.0.0.1	127.0.0.1	TCP	44	[TCP ZeroWindow] 58995 → 8888 [ACK] Seq=2730905484 Ack=252012209 Win=0 Len=0

数据包的 Payload 如下所示，符合发送的消息 Hello, server! I am client. From 202300460158.

0000	02 00 00 00 45 00 00 56 62 bc 40 00 80 06 00 00	E..V b.@....
0010	7f 00 00 01 7f 00 00 01 e6 73 22 b8 a2 c6 4f 5d	s"....0]
0020	0f 05 66 82 50 18 00 ff 8c ee 00 00 48 65 6c 6c	..f.P... ..Hell
0030	6f 2c 20 73 65 72 76 65 72 21 20 49 20 61 6d 20	o, serve r! I am
0040	63 6c 69 65 6e 74 2e 20 46 72 6f 6d 20 32 30 32	client. From 202
0050	33 30 30 34 36 30 31 35 38 2e	30046015 8.

四次挥手如下图所示：

746: 服务器发送 [FIN, ACK] (第一次挥手)。服务器 (端口 8888) 发送 FIN 包, 序列号 (Seq) 为 252012208, 确认号 (Ack) 为 2730905483, 表示主动请求关闭连接。

747: 客户端发送 [ACK] (第二次挥手)。客户端 (端口 58995) 回应服务器的 FIN, 发送 ACK, 确认号 (Ack) 为 252012209 (服务器的 Seq + 1), 表示确认收到服务器的关闭请求。

748: 客户端发送 [FIN, ACK] (第三次挥手)。客户端发送 FIN 包, 序列号 (Seq) 为 2730905483, 确认号 (Ack) 为 252012209, 表明客户端也无数据发送, 请求关闭连接。

753: 服务器发送 [ACK] (第四次挥手)。服务器回应客户端的 FIN, 发送 ACK, 确认号 (Ack) 为 2730905484 (客户端的 Seq + 1), 表示确认收到客户端的关闭请求。

tcp.port == 8888					
No.	Time	Source	Destination	Protocol	Length Info
739	15.353201	127.0.0.1	127.0.0.1	TCP	56 58995 → 8888 [SYN] Seq=2730905436 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
740	15.353275	127.0.0.1	127.0.0.1	TCP	56 8888 → 58995 [SYN, ACK] Seq=252012161 Ack=2730905437 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
741	15.353332	127.0.0.1	127.0.0.1	TCP	44 58995 → 8888 [ACK] Seq=2730905437 Ack=252012162 Win=65280 Len=0
742	15.353418	127.0.0.1	127.0.0.1	TCP	90 58995 → 8888 [PSH, ACK] Seq=2730905437 Ack=252012162 Win=65280 Len=46
743	15.353438	127.0.0.1	127.0.0.1	TCP	44 8888 → 58995 [ACK] Seq=252012162 Ack=2730905483 Win=65280 Len=0
744	15.353609	127.0.0.1	127.0.0.1	TCP	90 8888 → 58995 [PSH, ACK] Seq=252012162 Ack=2730905483 Win=65280 Len=46
745	15.353645	127.0.0.1	127.0.0.1	TCP	44 58995 → 8888 [ACK] Seq=2730905483 Ack=252012208 Win=65280 Len=0
746	15.353681	127.0.0.1	127.0.0.1	TCP	44 8888 → 58995 [FIN, ACK] Seq=252012208 Ack=2730905483 Win=65280 Len=0
747	15.353700	127.0.0.1	127.0.0.1	TCP	44 58995 → 8888 [FIN, ACK] Seq=2730905483 Ack=252012208 Win=65280 Len=0
748	15.353719	127.0.0.1	127.0.0.1	TCP	44 8888 → 58995 [ACK] Seq=252012209 Ack=2730905484 Win=65280 Len=0
753	15.663918	127.0.0.1	127.0.0.1	TCP	44 [TCP Retransmission] 8888 → 58995 [FIN, ACK] Seq=252012208 Ack=2730905484 Win=65280 Len=0
754	15.663953	127.0.0.1	127.0.0.1	TCP	44 [TCP ZeroWindow] 58995 → 8888 [ACK] Seq=2730905484 Ack=252012209 Win=0 Len=0

问题思考

1. 为什么第三次握手不需要携带数据？

TCP 三次握手的目的是建立可靠的连接, 确保双方通信的初始化序列号和确认号能够正确同步。第三次握手不需要携带数据是因为其用于连接的确认, 数据传输尚未开始, 因此无需携带实际数据。

2. 如果抓包中发现大量 ACK 报文但没有 PSH 标志, 可能是什么原因？

可能是双方在确认数据传输的状态,而当前没有新数据发送。也有可能接收方通过 ACK 调整窗口大小,通知发送方当前接收缓冲区状态导致只发 ACK。或者如果数据包丢失或超时,接收方可能反复发送 ACK 确认已接收的最后一个正确序列号,等待重传。

3. 如何通过 Wireshark 快速定位到四次挥手过程?

在 Wireshark 的过滤器栏输入 `tcp.port == 8888` (端口号), 筛选出 TCP 相关报文。四次挥手通常在连接末尾出现, 顺序为 [FIN]、[ACK]、[FIN]、[ACK]。在包列表中按时间排序, 找到连续的 FIN 和 ACK 组合。

此外, 四次挥手以 [FIN] 标志开始。还可以使用显示过滤器 `tcp.flags.fin == 1` 快速定位四次挥手中包含 FIN 的包。